
CONDORNET™
NEURAL MARKET INTELLIGENCE
CONDORBRAIN™
CONDORINTELLIGENCE™
TRADE SMART • TRADE AHEAD
CN v4.3 → v5.0
IMPLEMENTATION PLAN
FINAL STRETCH INTO LIVE ACCOUNT TRADING
v1.1 • PRECISION-GRADE
THE DOORWAY TO A NEW ERA OF INTELLIGENCE
AI
RiS
© 2026 by Quantor-MTFuzz™ - All rights reserved.

Confidential

Author: Dr. T. Jerry Mahabub, Ph.D.

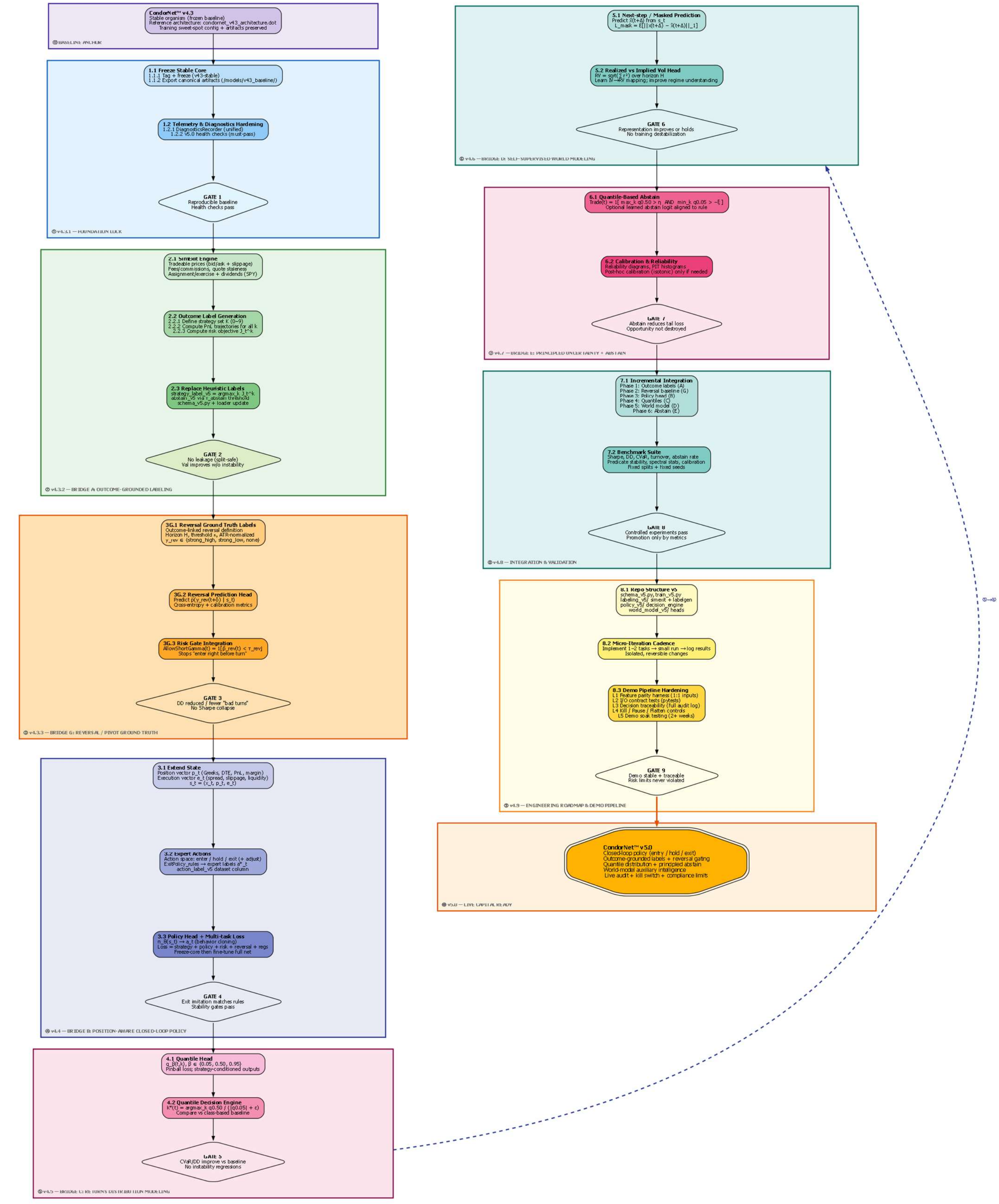

Table of Contents

The Following P0 →P4 Global principles (*apply everywhere*)

P0. No leakage. Any label computed from future data must be generated **only on train split**, then frozen and used as static labels for val/test.
P1. Single source of truth for execution rules. "*ExitRules*" must exist as a versioned module used by **SimExit**, **backtester**, **demo/live**.
P2. Identical strategy construction. Strategy ID $k$must map to exactly one canonical constructor across: labeling, training inference, backtest, live.
P3. Deterministic alignment. All feeds align on the same timestamp protocol (timezone, bar-close definition, missing bar handling).
P4. Every phase has a promotion gate. A phase is not "done" unless the benchmark suite passes.

---

v4.3.1 — Lock in the v4.3 foundation

1.1 Freeze the stable core

Task 1.1.1 — Tag & freeze CNv4.3

- Create branch `v43-stable` and git tag `cn-v4.3.0`.
- Freeze:
  - `intelligence/schema_v43.py`
  - `intelligence/condor_train_net_v43.py`
  - best-known config (CLI args + seed + data revision hash).

Task 1.1.2 — Export canonical artifacts

- Save under `/models/v43_baseline/`:
  - weights
  - training logs
  - predicate/set/superset convergence JSON
  - stability diagnostics
  - dataset manifest (file list + sha256 + column order hash).
- Add README with: *exact command line used, seed, git commit, dataset hash*.

DoD (Definition of Done)

- We can reproduce v4.3 run within tolerance (loss curves + stability stats) from scratch.

1.2 Telemetry & diagnostics hardening

Task 1.2.1 — Standardize diagnostics hooks

- Create `DiagnosticsRecorder` with:
  - spectral stats of L(or its parameterization)

    - clamp% and stability stats
    - predicate/set/superset churn
    - per-head losses, calibration metrics (later).
- Toggle via CLI flags and write JSONL per epoch.

Task 1.2.2 — Define v5.0 health checks
Minimum health checks:

- No runaway growth in dynamics (bounded state norms).
- Predicate/set/superset stability after N epochs.
- No loss collapse/divergence.
- Reproducible with fixed seed.

DoD

- A single command generates a "pass/fail" report artifact.

---

v4.3.2 — Bridge A: Outcome-grounded labeling (strategy labels tied to realized outcomes)

2.1 Build the forward P&L simulation engine (SimExit)

Task 2.1.1 — Implement `SimExit`
Inputs (minimum):

- timestamp t
- strategy ID k
- chain snapshot at entry
- underlying path and chain path over forward horizon
- commissions/fees model
- slippage model (bid/ask + liquidity regime)
- early assignment / exercise rules (at least for short ITM risk)
  Outputs:
- per-step P&L time series
- final realized P&L
- exit reason code (profit-take / stop / time / DTE / forced).

Critical corrections

- We must simulate using **tradeable prices**:
    - entry at ask for buys, bid for sells (or mid ± slippage)
    - exit likewise
- Must account for:
    - spread widening in stress

  - missing quotes / stale chain bars
  - corporate actions / dividends (SPY ex-div impacts early exercise risk).

Task 2.1.2 — Data alignment plumbing

- Create `MarketTape` abstraction:
  - `spot_bars[t]`
  - `chain_surface[t]`
- Enforce strict alignment rules:
  - if chain missing at t, either skip $t$or forward-fill with explicit flag (do not silently fill).

DoD

- Unit test: simulated entry/exit reproduces backtester results on a small sample window.

2.2 Generate outcome-grounded labels (per bar, per strategy)

Task 2.2.1 — Define strategy set $\mathcal{K}$

- Enumerate 0–9 strategies with explicit parameterization:
  - strike selection rules (delta targets, moneyness bands)
  - width rules
  - DTE bands
  - max contracts sizing rule (label-time only; live uses risk constraints).

Task 2.2.2 — Compute outcomes
For each bar $t$and strategy k:

$$\mathrm{PnL}_t^{(k)} = \mathrm{SimExit}(\mathrm{Enter}(t,k), \mathrm{ExitRules}, \text{future path})$$

Store:

- scalar realized P&L
- max adverse excursion (MAE)
- max favorable excursion (MFE)
- holding time
- exit reason.

Task 2.2.3 — Risk-adjusted objective
The current formula mixes signs in a way that can flip logic depending on conventions. Use an explicit "maximize profit, penalize tail loss" objective:

Option A (penalty form):

$$J_t^{(k)} = \mathbb{E}[\mathrm{PnL}_t^{(k)}] - \lambda \cdot \mathrm{CVaR}_\alpha(-\mathrm{PnL}_t^{(k)})$$

- $\mathrm{CVaR}_\alpha(-\mathrm{PnL})$measures **expected tail loss**.
- $\lambda \geq 0$is risk aversion.

Option B (ratio form, *if we prefer*):

$$J_t^{(k)} = \frac{\mathbb{E}[\mathrm{PnL}_t^{(k)}]}{\mathrm{CVaR}_\alpha(-\mathrm{PnL}_t^{(k)}) + \varepsilon}$$

Pick one; do not mix.

DoD

- For a sample period, label distribution makes sense (no single class dominates unless truly optimal).

2.3 Replace heuristic labels with outcome labels

Task 2.3.1 — Redefine strategy label

$$y_{\text{strategy}}(t) = \arg\max_{k \in \mathcal{K}} J_t^{(k)}$$

Task 2.3.2 — Redefine abstain

$$y_{\text{abstain}}(t) = \mathbf{1}\left(\max_k J_t^{(k)} < \tau_{\text{abstain}}\right)$$

Task 2.3.3 — Schema + dataset revision

- Add columns:
  - `strategy_label_v5`
  - `abstain_v5`
  - `J_best`
  - `J_all_k` (optional, or store in sidecar parquet)
  - `exit_reason_best`
- Create `schema_v5.py` (do not mutate v43 schema).

Promotion gate

- Training with v5 labels improves validation vs v4.3 baseline **without stability regressions**.

---

v4.3.3 — Bridge G: Reversal / pivot ground truth baseline (outcome-linked reversals + predictive head)

This was missing from the sequencing; it belongs **before** policy/exits because it becomes a universal risk gate.

3G.1 Build reversal ground truth label

Define forward excursion:

$$M_t^{(H)} = \max_{1 \le h \le H} \mid S_{t+h} - S_t \mid$$

Normalize:

$$R_t^{(H)} = \frac{M_t^{(H)}}{\mathrm{ATR}_t}$$

Directional validation:

- high pivot reversal if

$$\min_{1 \le h \le H} (S_{t+h} - S_t) < -\kappa \cdot \mathrm{ATR}_t$$

- low pivot reversal if

$$\max_{1 \le h \le H} (S_{t+h} - S_t) > \kappa \cdot \mathrm{ATR}_t$$

Label:

$$y_{\mathrm{rev}}(t) \in \{\mathrm{strong\_high,strong\_low,none}\}$$

3G.2 Add reversal prediction head

Train:

$$\hat{p}_\theta(y_{\mathrm{rev}}(t+\delta) \mid s_t)$$

Loss (cross-entropy) and calibration metrics.

3G.3 Use reversal probability as a hard risk gate

Example:

$$\text{AllowShortGamma}(t) = \mathbf{1}(\hat{p}_{\text{rev}}(t) < \tau_{\text{rev}})$$

Promotion gate

- Reversal head improves:
  - drawdowns
  - reduces “enter right before turn” events
  - does not degrade overall Sharpe.

---

v4.4 — Bridge B: Position-aware closed-loop policy (entry/hold/exit)

4.1 Extend state to include position + execution

Task 4.1.1 — Position vector p_t
Fixed-length encoding including:

- strategy ID
- strikes, widths, DTE, credit/debit
- Greeks exposure (net delta/gamma/vega/theta)
- margin usage
- unrealized P&L
- time in trade.

Task 4.1.2 — Execution vector e_t

- spread, liquidity flags, slippage estimate, quote staleness, fill quality.

Task 4.1.3 — Unified state

$$s_t = (x_t, p_t, e_t)$$

Hard requirement

- If no position open, $p_t$must be a well-defined “null position” vector with a mask flag.

4.2 Generate expert actions

Task 4.2.1 — Action space
Minimal:

- enter
- hold
- exit
  Optional later:
- adjust (roll/shift).

Task 4.2.2 — Rule expert

$$a_t^\star = \text{ExitPolicy}_{\text{rules}}(p_t, \text{PnL}_t, \text{risk}_t)$$

Task 4.2.3 — Dataset action labels

- Generate `action_label_v5` aligned to the same time grid.

Promotion gate

- Behavior-cloned exit matches rule exit on held-out data (confusion matrix + timing error).

4.3 Extend CondorNet with policy head

Task 4.3.1 — Add policy head

$$\pi_\theta(s_t) \rightarrow a_t$$

Task 4.3.2 — Multi-task loss

- outcome-grounded strategy loss
- action imitation loss
- risk head losses
- reversal loss (if Bridge G in)

Task 4.3.3 — Training regime

- Phase 1: freeze core, train heads
- Phase 2: unfreeze with small LR + stability gates

Promotion gate

- No stability regressions; exit decisions are consistent and reduce tail loss in sim.

---

v4.5 — Bridge C: Returns distribution modeling (quantiles)

5.1 Quantile head

Predict:

$$q_\beta(t,k) = \mathrm{QuantileNet}_\theta(s_t,k,\beta), \beta \in \{0.05,0.50,0.95\}$$

Pinball loss:

$$\mathcal{L}_\beta = \mathbb{E}\left[\rho_\beta\left(\mathrm{PnL}_t^{(k)} - q_\beta(t,k)\right)\right]$$

Implementation note

- Either condition quantiles on strategy embedding, or output all $k$in one shot with masking.

5.2 Decision rule

$$k^\star(t) = \arg\max_k \frac{q_{0.50}(t,k)}{\mid q_{0.05}(t,k) \mid +\varepsilon}$$

Promotion gate

- Quantile decisions improve risk metrics (CVaR/DD) vs class-only decisions.

---

v4.6 — Bridge D: Self-supervised world modeling (representation quality)

6.1 Next-step / masked prediction

$$\mathcal{L}_{\text{mask}} = \mathbb{E}[\parallel x_{t+\Delta} - \hat{x}_{t+\Delta} \parallel_1]$$

6.2 Realized vs implied volatility

The RV formula needs the square root (and typically annualization if used that way). Use:

$$\mathrm{RV}_{t:t+H} = \sqrt{\sum_{i=1}^{H} r_{t+i}^2}$$

(Optional scale factor for annualization if needed.)

Promotion gate

- Adds value without destabilizing training; improves calibration or downstream PnL.

---

v4.7 — Bridge E: Principled abstain (uncertainty + tail risk)

7.1 Quantile-based abstain gate

$$\mathrm{Trade}(t) = \mathbf{1}\left(\max_k\ q_{0.50}(t,k) > \eta\ \wedge\ \min_k\ q_{0.05}(t,k) > -\xi\right)$$

7.2 Calibration

- PIT histograms
- reliability diagrams
- post-hoc calibration only if needed

Promotion gate

- Abstain increases safety: lower DD / tail risk without killing opportunity.

---

v4.8 — Integration schedule & validation protocol

8.1 Incremental integration (The phases are correct, but enforce gates)

Phase order:

1. Outcome labels (A)
2. Reversal labels + head (G)
3. Policy head + position state (B)
4. Quantiles (C)
5. Self-supervised (D)
6. Principled abstain (E)

8.2 Benchmark suite (must be fixed)

Metrics:

- loss per head
- Sharpe, max DD, CVaR, turnover, abstain rate
- strategy distribution stability
- reversal calibration
- predicate stability
- dynamics stability stats

Hard rule

- Only promote when metrics improve or hold **and** stability gates pass.

---

v4.9 — Engineering roadmap & repo structure (tighten naming)

Create:

- `intelligence/schema_v5.py`
- `intelligence/labeling_v5/simexit.py`
- `intelligence/labeling_v5/labelgen_outcomes.py`
- `intelligence/labeling_v5/labelgen_reversal.py`
- `intelligence/policy_v5/decision_engine.py`
- `intelligence/world_model_v5/heads.py`
- `docs/plans/v5_implementation.md`
- `docs/architecture/v5_overview.md`

---

v5.0 — Demo → Real Live readiness checklist (corrected + de-risked)

Tighten this into **hard acceptance tests**:

L1. Feature parity harness

- A script that computes *exactly* the live feature vector and compares it to a recorded dataset slice:
  - column order
  - normalization
  - NaN handling
  - masks

L2. I/O contract tests

- Pytests for:
  - feed alignment
  - chain completeness
  - strategy constructor equivalence
  - SimExit vs backtester parity on sample windows

L3. Decision traceability

- Every decision must emit:
  - reversal prob

- chosen $k^\star$
- quantiles
- abstain gate evaluation
- predicate→set→superset top contributors
- risk envelope

L4. Kill/Pause/Flatten controls

- "Flatten All" and "Pause Trading" are mandatory.
- Post-mortem report generator is mandatory.

L5. Demo soak testing

- Run demo in real-time with monitoring and limits.
- Promote to live only after:
  - stable uptime
  - no schema mismatches
  - no uncontrolled order behavior
  - risk limits never violated.